The example in listing 7.4 binds an `int[]`, but you can bind an array of any simple type, including custom types with a `TryParse` method (listing 7.2), as well as `string[]` and `StringValues`.

NOTE `StringValues` is a helper type in the `Microsoft.Extensions.Primitives` namespace that represents zero, one, or many `string`s in an efficient way.

So where is that gray area I mentioned? Well, arrays work as I've described only if

- You're using an HTTP verb that typically doesn't include a request body, such as `GET`, `HEAD`, or `DELETE`.
- The array is an array of simple types (or `string[]` or `StringValues`).

If either of these statements is *not* true, ASP.NET Core will attempt to bind the array to the JSON body of the request instead. For `POST` requests (or other verbs that typically have a request body), this process works without problems: the JSON body is deserialized to the parameter array. For `GET` requests (and other verbs without a body), it causes the same unhandled exception you saw in figure 7.4 when a body binding is detected in one of these verbs.

NOTE As before, when binding body parameters, you can work around this situation for `GET` requests by adding an explicit `[FromBody]` to the handler parameter, but you shouldn't!

We've covered binding both simple types and complex types, from the URL and the body, and we've even looked at some cases in which a mismatch between what you expect and what you receive causes errors. But what if a